SolarSabha

Technical Guide: how the whole system works

SolarSabha is an AI project manager for a housing society's rooftop solar project under **PM Surya Ghar: Muft Bijli Yojana**. It reads the society's electricity bills and vendor quotations, sizes the system, audits the quotations against the scheme's rules, prepares meeting papers and letters, and follows up with the DISCOM (the electricity company) for weeks. People approve every decision; the app cannot send, sign, pay or file anything.

Live site	https://solarsabha.d2vnrqt0vyz925.amplifyapp.com
API	https://f9x2xp42wg.execute-api.ap-south-1.amazonaws.com
Code	https://github.com/kh-bikash/solarsabha
AWS	CloudFormation stack **solarsabha**, region ap-south-1 (Mumbai)
Built for	WeMakeDevs x AWS Environmental Hacks, Bharat Builds Tour 2026, Track 03: Waste and Energy
This edition	11 October 2026

Contents

1. The big picture
2. One request, start to finish
3. The data: one record per project
4. The plain-code core: numbers and rules
5. Reading documents
6. The AI agents
7. Models, quotas and time limits
8. Long-running work: reviews and alarms
9. The frontend
10. Security and privacy
11. Configuration reference
12. Deploying and operating it
13. Testing and quality
14. What happens when something fails
15. Limits and next steps
16. Glossary

1. The big picture

The app has three layers. The **browser** runs a React app (the pixel-art office and its folders). **AWS** runs a small serverless backend: one API function that does all the work, one wake-up function for scheduled follow-ups, and managed storage. **AI models** are called by the API function only for reading, explaining and wording; every number comes from plain Python.

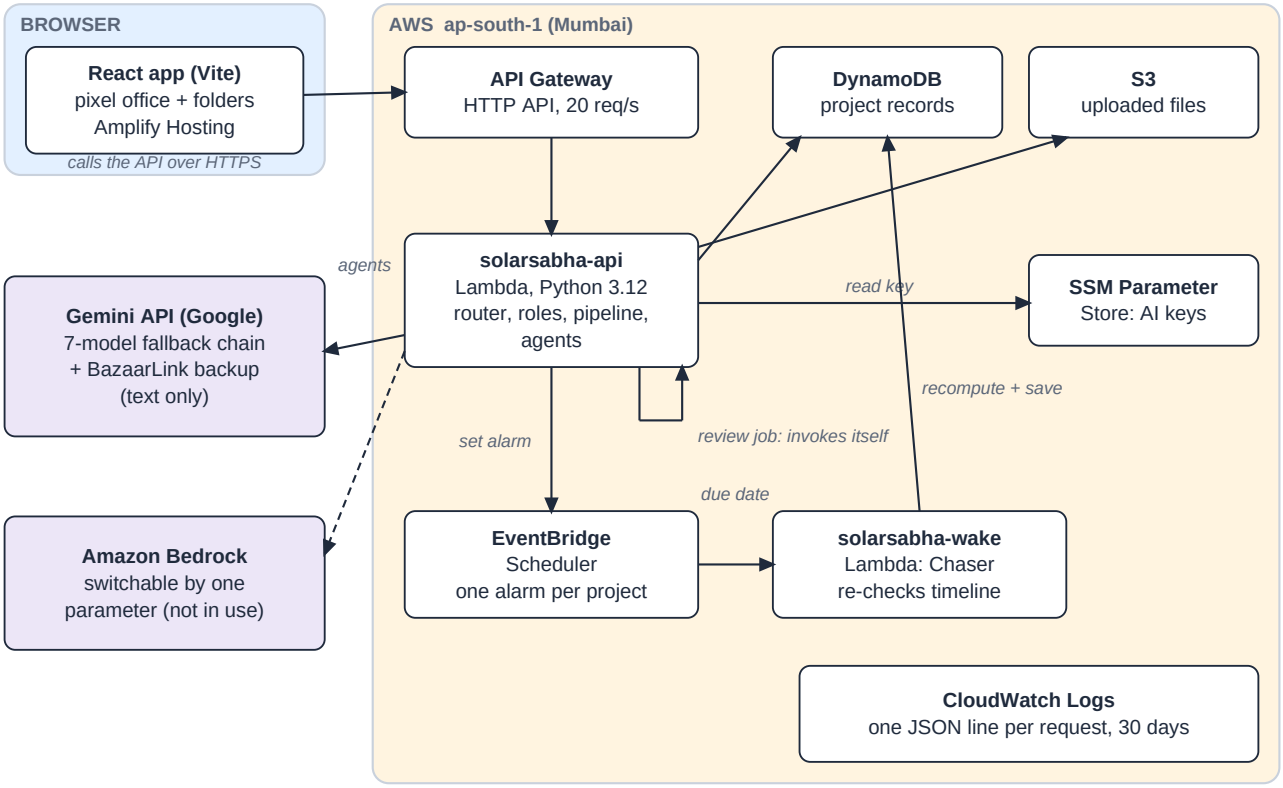


Figure 1. Components and how they talk to each other.

Part	What it does	AWS name
Amplify Hosting	Serves the built React app (static files)	app SolarSabha, branch solarsabha
API Gateway (HTTP API)	Public HTTPS entry; CORS for the site only; throttled to 20 req/s (burst 50)	f9x2xp42wg
API Lambda	Routing, access keys, roles, validation, uploads, pipeline, agents, Word export	solarsabha-api
Wake Lambda	Runs when a project's alarm rings; re-checks the timeline; sets the next alarm	solarsabha-wake
DynamoDB	One item per project (the whole state as JSON), versioned	solarsabha-StateTable-...
S3	Original uploaded files, private and encrypted	solarsabha-uploadsbucket-...
EventBridge Scheduler	One-time alarm per project for the Chaser	solarsabha-<project id>
SSM Parameter Store	AI keys, encrypted (SecureString)	/solarsabha/gemini-api-key
CloudWatch Logs	Request logs, 30-day retention	/aws/lambda/solarsabha-api
CloudWatch Alarms + SNS	Six alarms for errors, failed reviews, throttles and 5xx; email via SNS	solarsabha-* alarms, AlertsTopic

Everything on AWS is defined in one file, `infra/template.yaml` (AWS SAM). Nothing was clicked together by hand except the Amplify app and the SSM key.

2. One request, start to finish

Take **PUT /projects/{id}/bills** (someone edits a bill). The same path applies to every route.

Step	What happens	Where
1	The browser sends JSON with two headers: **x-project-key** (the project's secret) and **x-actor** (the committee member's name)	frontend/src/api.js
2	API Gateway checks CORS and the rate limit, then calls the Lambda (payload v2)	template.yaml
3	The handler matches the route in a table of (method, path, function, who may call)	handlers/api.py ROUTES
4	The project is loaded from DynamoDB with a strongly consistent read; expired projects read as missing	state.py
5	The access key is hashed (SHA-256) and compared in constant time with the stored hash	project.check_key
6	The actor must be a member with the right role: read = anyone with the key; edit = secretary or member; secretary-only for members and delete	project.require_role
7	The body is validated with Pydantic (types, limits; 9 MB maximum)	schemas.py
8	The change is applied, logged in the project's activity log, and the whole pipeline is re-run	project.recompute
9	The Chaser's alarm is moved to the new due date	alarm.sync
10	The project is saved with a version check (see section 3)	state.DynamoStore.put
11	The response is the full project, minus the key hash; one JSON log line is written	handlers/api.py

Errors the browser can get

Code	Meaning
400	Invalid input (the message lists the fields)
401	Missing or wrong access key
403	Not a member, or the role cannot do this
404	Unknown or expired project / route
409	Someone else saved a moment ago (the page reloads), or a review is already running
413	Request too large
422	AI polishing changed a fact, so it was rejected
502	No AI model could answer; nothing was changed
500	Unexpected error; logged with a request id

All routes

```
GET    /health                              GET  /rules  (rulebook + states)
POST   /projects                            {"sample": true} or society + you + consent; returns the key ONCE
GET    /projects/{id}       DELETE /projects/{id} (secretary, confirm name)     GET .../export
PUT    .../society | bills | quotes | flats | assumptions | events | members
POST   .../documents        GET/DELETE .../documents/{did}
POST   .../discom-reply     DELETE .../discom-reply
PUT    .../drafts/{did}     POST .../drafts/{did}/approve     GET .../drafts/{did}/docx
POST   .../advance {days}   POST .../clock/reset          (simulated clock for the demo)
POST   .../review           POST .../drafts/{did}/polish  (live AI mode only)
```

3. The data: one record per project

Each project is a single DynamoDB item: `pk = PROJECT#<id>`, `sk = STATE`. The whole project is stored as one JSON document in the `doc` attribute, next to a `version` number and a `ttl` for automatic deletion. One item per project keeps every save atomic.

Field	Content
id, schema, created_at, updated_at	10-hex-character id; record format version
key_hash	SHA-256 of the access key. The key itself is shown once and never stored
consent, expires_at	Privacy consent (notice version 2026-10c) and the retention deadline
members	Names and roles: secretary, member, viewer
inputs	Society details, bills, quotes, flat areas, AGM date
assumptions	Committee overrides: tariff, cost per kW, yield, share of load
documents	Each upload: status, which reader, notes, fields read, S3 name
events, started_on, sim_offset_days	Real dates each stage was done; the demo's simulated offset
discom_reply, reply_reading	The pasted DISCOM email and how it was read
draft_state	Edits and approvals, keyed by a stable draft id, with content hashes
review, review_job	The agents' last review and the background job's status
audit	Activity log: who did what and when (last 500 entries)
result	The latest pipeline output: plan, audit, timeline, drafts, trace, rulebook
alarm_at, version	Next Chaser wake-up; the optimistic-lock counter

Versioned saves (optimistic locking)

Two things can change a project at the same time: a person's edit and a background review that takes minutes. Without protection, the slower one would silently overwrite the other. So every record carries a **version**. A save says "write this only if the stored version is still N" (a DynamoDB conditional write: `ConditionExpression: version = :v`) and stores N+1. If someone saved in between, the write is refused with **ConflictError**:

- A person's request gets **409**; the page reloads the project and asks them to try again.
- The background review and the wake-up re-read the latest record and retry (up to 5 times), so their result is merged instead of lost.
- Locally the same rule is applied to JSON files, so tests catch lost updates too.

Files and retention

Originals go to S3 as `<project id>/<doc id>-<safe name>`. Projects expire after **365 days** (RetentionDays): DynamoDB TTL deletes the item and an S3 lifecycle rule expires the files. Deleting a project removes its item, its files and its alarm at once. Point-in-time recovery is on for the table.

4. The plain-code core: numbers and rules

`pipeline.py` is the heart of the product: a **pure function** from inputs (bills, quotes, events, assumptions, DISCOM reply) to a result (plan, audit, fairness table, timeline, next action, letters, trace). It uses no AI and no AWS, runs in milliseconds, and is re-run after every change. The AI never does arithmetic; it calls these functions.

The rulebook

`data/rulebook.json` holds every rule with its value, source, status (*official*, *verify*, *team assumption*) and date, plus state overrides. The app shows this citation next to every flag. Key values:

Rule	Value	Status
Subsidy for a society's common facilities (GHS)	Rs 18,000 per kW (Rs 19,800 in special category states)	official
Subsidy ceiling	3 kW per flat, 500 kW per society	official
Vendor must be registered on the National Portal	otherwise no subsidy	official
Free maintenance contract (CMC)	at least 5 years	official
Subsidy (CFA) paid within	15 days of DISCOM approval	official
Grid emission factor (CEA)	0.71 t CO2 per MWh	official
Yield	1,400 kWh per kW per year	verify
Share of load to cover; cost; tariff	85%; Rs 50,000/kW; Rs 7/kWh	team assumption
Advance payment flag	above 20%	team assumption

Sizing and money

```
annual_kwh  = sum of the last 12 bills' units
ceiling     = min(flats x 3 kW, 500 kW)            (or the DISCOM's limit after a reply)
kW          = round(annual_kwh x share / 1400), clipped to 1..ceiling
              options: small (60% of load), recommended (85%), large (100%)
cost        = kW x cost_per_kW          subsidy = kW x 18,000
saving/year = kW x 1400 x tariff        payback = (cost - subsidy) / saving
CO2/year    = kW x 1400 / 1000 x 0.71 tonnes
```

For the sample society this gives 36 kW: Rs 18 lakh cost, Rs 6.48 lakh subsidy, Rs 11.52 lakh net, about 3.3 years simple payback and about 36 tonnes of CO2 avoided a year.

Quote audit

`tools/quotes.py` checks each quotation and gives a verdict: **RED** if any red flag, **AMBER** if any flag, else **GREEN**. Missing fields are flagged, never guessed.

Flag	Level	Why
VENDOR_NOT_VERIFIED	red	Not on the registered vendor list: the subsidy would be refused
DOCUMENT_CONTAINS_INSTRUCTIONS	red	Hidden or visible text tries to instruct the AI (prompt injection)
PRICE_LOW / PRICE_HIGH / PRICE_MISSING	amber	Price per kW outside Rs 42,000 to 58,000, or no price
ADVANCE_HIGH / ADVANCE_MISSING	amber	Advance above 20%, or not stated
PAY_BEFORE_APPROVAL	amber	Asks for payment before DISCOM approval (subsidy risk)
PRESSURE	amber	Pressure tactics such as offers that expire today
SERVICE_SHORT / SERVICE_MISSING / WARRANTY_MISSING	amber	Maintenance under 5 years, or terms missing

Timeline, fairness and letters

- **Timeline:** six stages (AGM, portal, DISCOM feasibility, installation, net meter, subsidy) with expected durations from the rulebook. Each stage is done, active, upcoming or **overdue**. Overdue by 1 to 7 days means a reminder letter; more than 7 days means an escalation.
- **Fairness:** who pays what, split equally or by flat area.
- **Letters:** AGM notice, resolution, vendor clarification, DISCOM reminder and escalation, filled from the facts. Editing a letter after approval withdraws the approval (content hash check).
- **DISCOM reply:** a rule classifier spots a capacity limit ("can permit only upto 30 kW"), approval or a documents request; a limit produces **Plan v2** at the new size.

5. Reading documents

Status of each document: READ (clean) | NEEDS_REVIEW (unclear, provisional, or the two readers disagree) | NEEDS_ENTRY (nothing readable; a person types the values). If the AI read fails for any reason the rule reader answers, so an upload is never lost.

Every value read can be corrected in Documents before it is used, and the plan shows which bill each number came from.

Figure 2. What happens to an uploaded bill or quotation.

Two readers

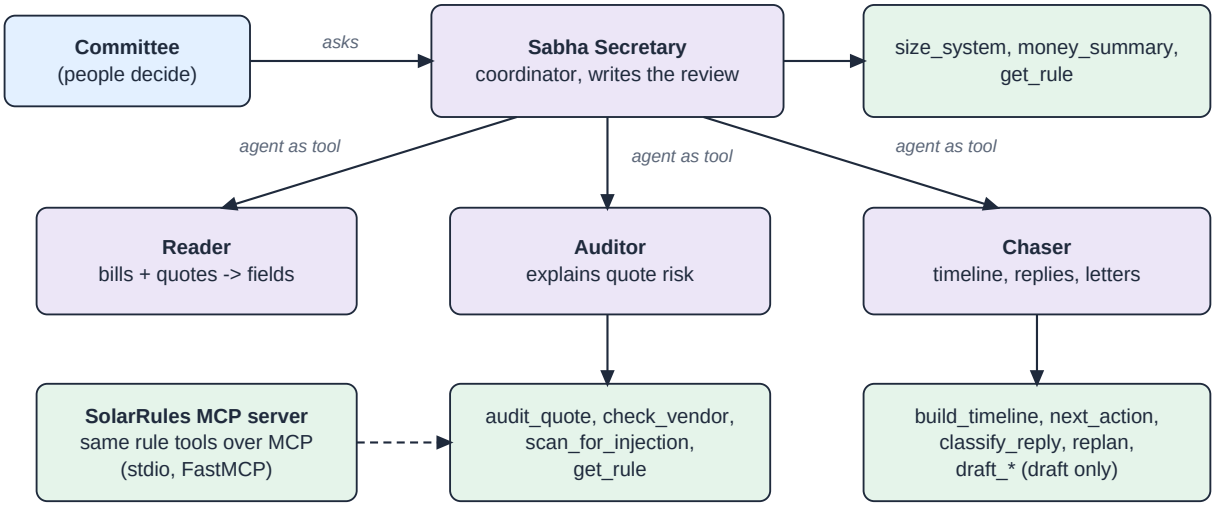
	Offline reader (reading.py)	AI reader (agents/reader.py)
Works on	Digital PDFs (text layer)	PDFs, photos and scans
How	pypdf extracts all text, including hidden text; regular expressions find month, units, amount, vendor, price, terms	Gemini reads the file as a document or image and fills a typed form (BillData / QuoteData)
When	Always available; the only reader when READER_MODE=offline	READER_MODE=live; 20 s budget per upload
Cost	Free, instant	One model call

Safety checks on an AI reading

- **Cross-check:** on a digital PDF, the AI's numbers are compared with the offline reader's. Any disagreement marks the document NEEDS_REVIEW with a note naming the fields.
- **Provisional bills:** a bill marked provisional (meter not read, units estimated) is flagged.
- **Month format:** months are normalised (2026-09, Sep-2026, 09/2026 all become 2026-09).
- **Hidden text:** the offline reader's full text, including white or tiny text, is added to the quote so the injection scan sees what the AI might have skipped.
- **Documents are data:** prompts tell every agent that document text is data, never instructions, and an instruction-like phrase turns the quote red.

6. The AI agents

The agents are built with the **Strands Agents SDK** (strands-agents 1.58). Each agent is a model plus a system prompt plus a small set of tools. They read, explain and draft; they never calculate and cannot act outside the app.



Purple = AI agents (Strands). Green = plain, tested Python. Agents explain and draft; code does every number and every rule check.

Figure 3. The agent team and the tools each one may use.

Agent	Job	Tools it may call
Sabha Secretary	Coordinates; writes the committee's review (summary, a question per vendor, next step, what needs a decision, what is not verified)	reader, auditor, chaser (agents as tools), size_system, money_summary, get_rule
Reader	Turns a bill or quote into fields	none (reads the file it is given)
Auditor	Runs the quote audit and explains each flag in plain words	audit_quote, check_vendor, scan_for_injection, get_rule
Chaser	Checks the timeline, reads DISCOM replies, drafts follow-ups, re-plans	build_timeline, next_action, classify_reply, replan_for_load_limit, draft_discom_reminder, draft_discom_escalation, get_rule

How a review runs

- The Secretary gets the project facts as JSON, marked as data. It asks the Auditor about each quote, then the Chaser about the timeline, and returns a typed **LiveReview**.
- **Agents as tools:** each specialist is exposed to the Secretary as a tool. A fresh specialist is built for every call, on the same model the Secretary is using, because a shared agent refuses a second call while busy.
- **One at a time:** the Secretary uses Strands' SequentialToolExecutor, so specialist calls never overlap and the trace stays in order.
- **Typed answers:** every agent returns a Pydantic model (LiveReview, ReplyReading, PolishedLetter, BillData, QuoteData). Strands delivers it through a special answer tool.

Guardrails (trace.py, TraceHooks)

- Before any tool runs, a Strands hook checks it against that agent's allowlist and **cancels** anything else, recording "Blocked a call" in the trace. The agent's own answer tool is always allowed.
- After a tool runs, its real input and output go into the trace. The office animation and the Logbook replay these real events, not a script.

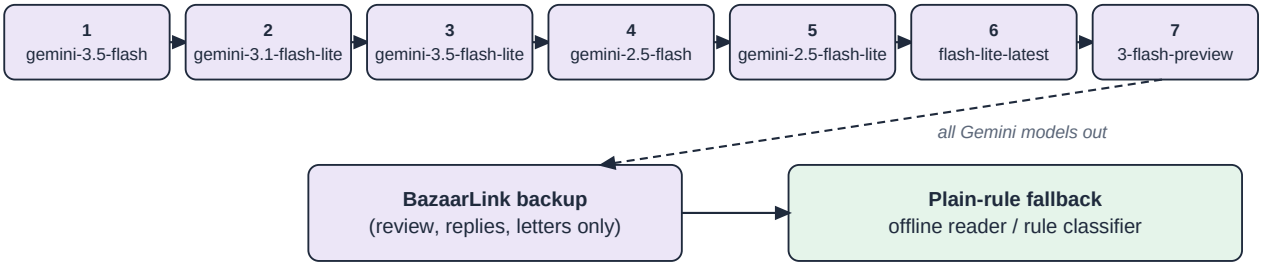
- There is no send, sign, pay or file tool anywhere, so no prompt can make an agent do those things.
- **Polish (AI)** rewords a letter with a tool-less writer agent; facts_unchanged() compares every number, date and reference before and after, and rejects the result (422) if anything changed.

SolarRules MCP server

`backend/mcp_server/solar_rules_server.py` serves the rulebook and calculators (get_rule, list_rules, size_system, money_summary, check_vendor, scan_for_injection) over the Model Context Protocol (FastMCP, stdio). During a review the server is started as a subprocess and its tools replace the built-in ones of the same name. If it cannot start in a container, the review continues with the built-in tools and says so; it is not retried for every model.

7. Models, quotas and time limits

MODEL_PROVIDER picks the model behind every agent: **gemini** (default) or **bedrock**. Agents, tools, hooks and prompts are identical either way; only factory.model() changes. Free model quotas are small, and API Gateway gives up after 30 seconds, so calls go through `run_with_fallback()`:



Per model: daily quota used -> skip it for the life of the container. Per-minute limit -> wait the time Google states (max 5 s inside a request, 30 s in the background), retry once, then move on. Overloaded (503) or a garbled tool call -> next model. Any other error (bad key, bad request) is raised at once: retrying would not help. Uploaded documents are never sent to the backup.

Figure 4. The model chain. Quota and overload errors move down; real errors stop.

Time budgets

Where	AI time allowed	If it runs out
Reading an upload	20 s	The offline reader answers, with a note
Reading a DISCOM reply	20 s	The rule classifier answers
Polishing a letter	20 s	502 "did not answer in time"; the letter is unchanged
The agents' review (background)	480 s (Lambda timeout 540 s)	review_job = failed with the reason

A budgeted call runs in a worker thread and is abandoned when its time is up, so the request can still answer before API Gateway's 30-second cut-off.

Error handling details

- Errors are classified by walking the whole exception chain, because the useful detail (for example Google's "PerDay" quota id) sits in the underlying error, not in Strands' wrapper.
- Strands' own retries are kept short for Gemini (2 attempts, 2 to 4 s) so a used-up model is abandoned quickly; backups get 4 slower attempts.
- If a specialist runs out of quota mid-review, it cancels the Secretary and the error reaches run_with_fallback, so the whole review restarts on the next model instead of looping.
- Used-up models are remembered per container (a warm Lambda), so later requests skip them at once.

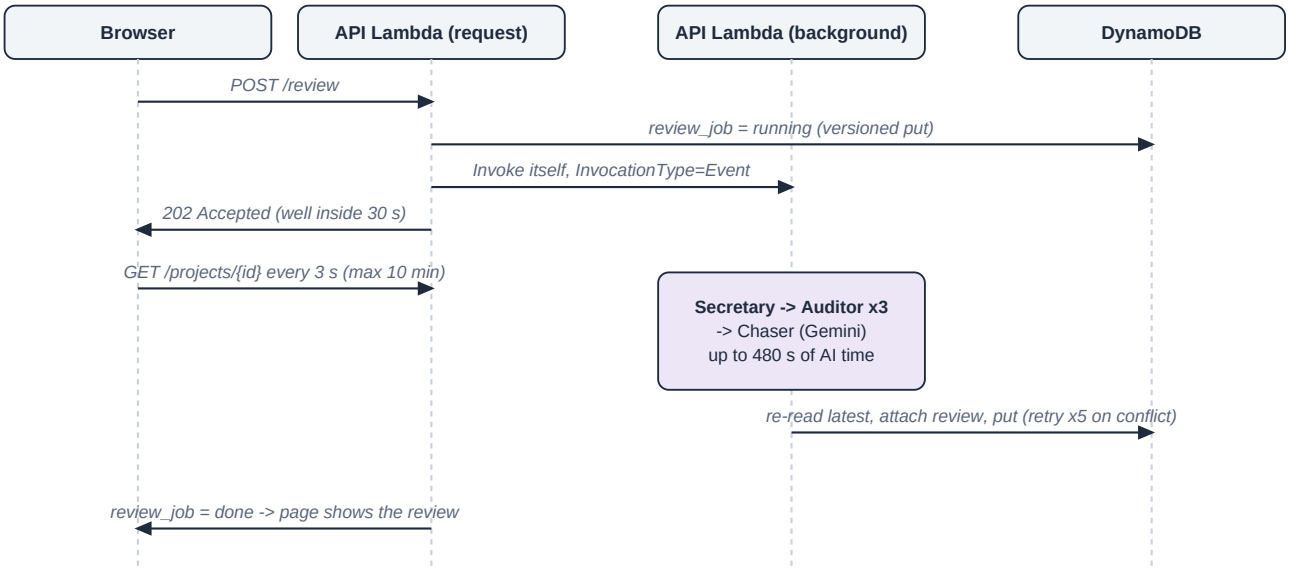
Where the keys live

Locally: `.env` (git-ignored): GEMINI_API_KEY, optional BAZAARLINK_API_KEY. On AWS: encrypted SSM parameters (`/solarsabha/gemini-api-key`, optional `/solarsabha/bazaarlink-api-key`). The Lambda reads them at run time and may read only `/solarsabha/*`. Keys never appear in the template, environment variables, code, logs or the browser.

8. Long-running work: reviews and alarms

The agents' review runs in the background

A full review (Secretary, three Auditor calls, Chaser) takes one to three minutes, but API Gateway answers within 30 seconds. So on AWS the API Lambda starts the review as an **asynchronous invocation of itself** and returns at once; the page polls. Locally (dev server) it simply runs inside the request.

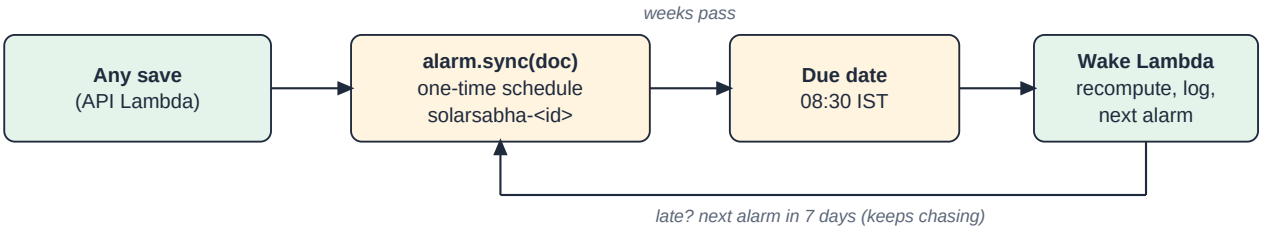


If the AI fails, review_job = failed with a clear message. A job still 'running' after 600 s is treated as dead, so a new review can start. Lambda's automatic retry of failed async calls is off (MaximumRetryAttempts: 0), so a review never runs twice by itself.

Figure 5. The background review job.

The Chaser's alarm clock

An agent cannot sit and wait for three weeks, and nothing should run (or be billed) while waiting. So the Chaser sets an alarm instead:



Nothing runs and nothing is billed while waiting. Deleting a project deletes its schedule. A scheduler error is logged and never blocks a save.

Figure 6. How the Chaser follows up for weeks.

The demo's **time machine** (POST /advance) moves a separate, clearly labelled offset (sim_offset_days) through the very same recompute, so you can watch a 16-days-late escalation appear without waiting. Reset returns to the real date.

9. The frontend

React 18 built with Vite, in `frontend/src`. No UI framework: hand-made pixel-art SVG sprites and plain CSS, themed for Track 03.

Piece	What it does
api.js	All server calls. LIVE mode when VITE_API_URL is set at build time; MOCK mode (no server, the bundled sample, read-only) otherwise. A 409 conflict fires an event that reloads the project.
session.js	Remembers this browser's projects (id + key) in localStorage. A share link carries the key in the URL hash (#p=<id>&k=<key>), which browsers never send to servers.
office/ and pixel/	The office scene. story.js turns real trace events into sentences and tells the scene which agent walks where and which object lights up (calculator, filing cabinet, calendar).
components/	The folders: Welcome, Documents, PlanCard, QuoteTable, Fairness, Timeline, Drafts (your desk), Arena (negotiation simulation), TracePanel (logbook and the agents' review), Settings, PrivacyNotice.
App.jsx	Loads the project, plays new trace events, runs actions, polls the review job every 3 s (up to 10 minutes), and shows which model family is live in the header.

Build for production: `VITE_API_URL=<ApiUrl> VITE_PRIVACY_CONTACT="..." npm run build`, then upload `dist/` to Amplify.

10. Security and privacy

Concern	How it is handled
Who can open a project	Only holders of its link: a random 32-character key; only its SHA-256 hash is stored; compared in constant time
Who can change what	Roles per member: secretary (everything), member (edit, approve), viewer (read)
Uploads	6 MB cap, 60 per project, type decided by magic bytes (PDF/PNG/JPEG), file names sanitised
Storage	DynamoDB and S3 encrypted at rest; bucket private, TLS-only, public access blocked
API	CORS limited to the site; rate limited; no-store and nosniff headers; key hash never returned
Least privilege	Each Lambda gets only the permissions it needs (table, bucket, its schedules, /solarsabha/* parameters, invoking itself)
Logs	One JSON line per request: route, status, time, project id. No document content, no names
Consent and rights	Privacy notice accepted at creation; export, correction and deletion built in; auto-deletion after a year (DPDP Act, 2023)
AI data	Gemini processes data outside India; the free tier may be used to improve Google's products, so real society data needs a paid key or Bedrock's India-only profile. The backup never receives documents
Actions	Draft only: no send, sign, pay or file code exists; approvals are named and withdrawn on edit

11. Configuration reference

Environment variables (backend)

Variable	Meaning	Set by
READER_MODE	live = AI agents on; anything else = offline reader, no AI	.env / template
MODEL_PROVIDER	gemini (default) or bedrock	.env / template
GEMINI_API_KEY / GEMINI_KEY_PARAM	Key directly (local) or the SSM parameter name (AWS)	.env / template
GEMINI_MODEL_ID	Comma-separated model chain (default: 7 models)	template
BAZAARLINK_API_KEY / BAZAARLINK_KEY_PARAM	Optional text-only backup	.env / template
BEDROCK_MODEL_ID	Model or inference profile when MODEL_PROVIDER=bedrock	.env / template
STATE_BACKEND	dynamodb on AWS; local JSON files otherwise	template
TABLE_NAME, UPLOADS_BUCKET	Where state and files go	template
SCHEDULER_ROLE_ARN, WAKE_FUNCTION_ARN	Turn on the Chaser's alarms	template
ALLOWED_ORIGIN	The site allowed by CORS	template
RETENTION_DAYS	Days before a project is deleted (365)	template

Template parameters (sam deploy)

Parameter	Current value
ReaderMode	live
ModelProvider	gemini
GeminiKeyParam	/solarsabha/gemini-api-key
GeminiModelChain	the 7-model default
BazaarlinkKeyParam	empty (backup off on AWS)
BedrockModelId	in.anthropic.claude-sonnet-5 (unused while provider is gemini)
AllowedOrigin	https://solarsabha.d2vnrqt0vyz925.amplifyapp.com
RetentionDays	365
AlarmEmail	empty (alarms visible in CloudWatch; no email yet)

12. Deploying and operating it

```
# backend (Docker Desktop running; --use-container avoids a Windows-only MCP dependency)
cd infra
sam build --use-container
sam deploy --stack-name solarsabha --region ap-south-1 --resolve-s3 --capabilities CAPABILITY_IAM \
  --parameter-overrides ReaderMode=live ModelProvider=gemini \
  GeminiKeyParam=/solarsabha/gemini-api-key \
  AllowedOrigin=https://solarsabha.d2vnrqt0vyz925.amplifyapp.com

# frontend
cd frontend
VITE_API_URL=https://f9x2xp42wg.execute-api.ap-south-1.amazonaws.com npm run build
# zip dist/, then: aws amplify create-deployment -> upload zip -> aws amplify start-deployment

# rotate the AI key (no redeploy needed; a new Lambda container picks it up)
aws ssm put-parameter --region ap-south-1 --type SecureString --overwrite \
  --name /solarsabha/gemini-api-key --value "NEW_KEY"

# remove everything
sam delete --stack-name solarsabha --region ap-south-1
aws amplify delete-app --app-id d2vnrqt0vyz925 --region ap-south-1
```

Where to look in the AWS Console (region Mumbai)

To see	Open
Every resource	CloudFormation > solarsabha > Resources
Project records	DynamoDB > Tables > solarsabha-StateTable-... > Explore table items
Uploaded files	S3 > solarsabha-uploadsbucket-...
Errors and timings	CloudWatch > Log groups > /aws/lambda/solarsabha-api
Chaser alarms	EventBridge > Scheduler > Schedules (solarsabha-<id>)
Health alarms	CloudWatch > Alarms > All alarms (solarsabha-*)
The AI key	Systems Manager > Parameter Store

Alarms

Six CloudWatch alarms send to one SNS topic. Handled errors (bad input, wrong key, a 502 when no AI model can answer) are answered by the API itself, so two log metrics pick out the real problems from the request log.

Alarm	Fires when
solarsabha-unhandled-errors	An unhandled exception (log metric on unhandled_error), 1 in 5 minutes
solarsabha-review-failures	3 or more failed agent reviews in an hour (usually every model out of quota)
solarsabha-api-lambda-errors	The API Lambda crashed or timed out
solarsabha-api-lambda-throttles	Lambda refused requests for lack of concurrency
solarsabha-wake-lambda-errors	A scheduled Chaser wake-up failed
solarsabha-api-5xx	10 or more server errors in 5 minutes at API Gateway

For email, redeploy with `AlarmEmail=you@example.com` and click the confirmation link AWS sends. Missing data counts as OK, so a quiet night raises nothing.

Cost: at hackathon traffic everything sits in free allowances (Lambda, API Gateway, on-demand DynamoDB, S3, Scheduler, standard SSM parameters). Set a zero-spend budget alert in Billing so any charge emails you.

13. Testing and quality

Check	Command	Result
Backend unit and API tests: reader, pipeline, roles, privacy, alarms, wake-up, locking, model fallback, guardrails, live agents with fake models	make test	87 passing
Frontend tests (Vitest)	make test-ui	32 passing
Accuracy scoreboard: tools + offline reader on 18 specimen PDFs with known answers	make evals	30/30
AWS template lint	make lint-infra	clean
Live end-to-end on AWS: AI bill reading, background review, DISCOM reply, polishing	manual script	passing
Backend tests, evals, template lint, frontend tests and build, on every push	GitHub Actions	green

Agent tests replace the model with fakes that call the real tools, so the wiring, guardrails, structured answers and fallbacks are tested without spending quota. The specimen documents in `backend/samples/pdfs` are fictional and watermarked SPECIMEN; they include a provisional bill, a quote with no price and a quote with a hidden instruction.

14. What happens when something fails

Failure	What the user sees	Data
Gemini daily quota used up	Nothing: the next model in the chain answers	safe
Every model out of quota	Uploads read by the offline reader with a note; review fails with a clear message; plan, audit, letters, timeline unaffected	safe
AI slower than 20 s in a request	The rule-based answer, with a note	safe
AI reads a number differently from the rules	Document marked Needs review, naming the fields	safe
Two people save at once	The second gets a reload and tries again	safe (versioned)
Review job's Lambda dies	After 10 minutes a new review can be started	safe
MCP server cannot start	Review runs with built-in tools; noted in the trace	safe
Scheduler error	Logged; the save still succeeds	safe
A quote hides instructions for the AI	Quote turns RED with the reason	safe
Unexpected bug	"Something went wrong" with a request id to find in the logs	unchanged

15. Limits and next steps

Limit	Next step
Free Gemini tier: about 20 requests per model per day; content may be used by Google	A paid key (pennies per review), or Bedrock with the India-only profile once the AWS account allows it
Access is by link, and people pick their own name	Amazon Cognito sign-in in front of the API
Alarms email nobody until an address is set	Redeploy with AlarmEmail and confirm the link AWS sends
Only national and special-category rules verified	Add each state's DISCOM rules once checked
Vendor list, negotiation replies and the sample DISCOM reply are simulated	Live vendor registry lookup; real email threads
Not load-tested	A short load test before a real launch

16. Glossary

Term	Meaning
PM Surya Ghar	The national rooftop solar scheme (Muft Bijli Yojana) that pays the subsidy
GHS / RWA	Group Housing Society / Resident Welfare Association: a society's common facilities
CFA	Central Financial Assistance: the subsidy itself
DISCOM	The electricity distribution company: checks feasibility, fits the net meter, inspects
Feasibility	The DISCOM's check that the local network can take the solar system
Net meter	A meter that counts electricity both ways, so exported solar is credited
AGM / SGM	The society's general body meeting, where the project is approved
kW / kWh	System size / energy. 1 kW of panels makes about 1,400 kWh a year here
CMC	Comprehensive maintenance contract: 5 years free, required by the scheme
National Portal	pmsuryaghar.gov.in, where vendors register and societies apply
Strands Agents	AWS's open-source SDK for building AI agents with tools
MCP	Model Context Protocol: a standard way to serve tools to AI agents
SAM	AWS Serverless Application Model: the template that creates the AWS resources
Optimistic locking	Save only if nobody else saved since you read; otherwise retry